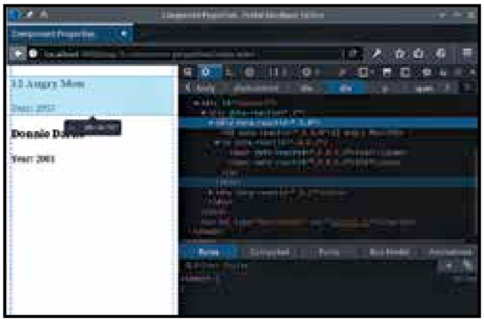


If you look into the actual DOM at this point you will see a lot of the stuff that react has added to keep track.

```
 render: function() {
   return (
     <div onClick={this.handleClick}>{this.star()}</
div>
   );
 }
});

ReactDOM.render(<Star />, docu-
ment.getElementById('content'));
```

There is a lot going on here, so let's break it down.

The getInitialState method of Star returns the initial state of the component. The render method is a simple div that has a click handler which we have bound to the handleClick method, and the body of the div is returned by the method star.

The handleClick method is simple. It just toggles the starred state between true / false. Using setState causes the component to automatically be re-rendered with the new state.

The star method returns the Unicode symbol for a full star or empty star based on the starred state variable.

We also render the Star component on the page. This should display a single empty star that toggles between full / empty when clicked.

You could now include <Star /> in your Movie component and get this component as part of the Movie component. Right now this isn't useful because the Movie component doesn't know the state of the star. This problem is fixable and hopefully when you're done reading you'll know how.

Right now though let's start by creating a MovieList component.

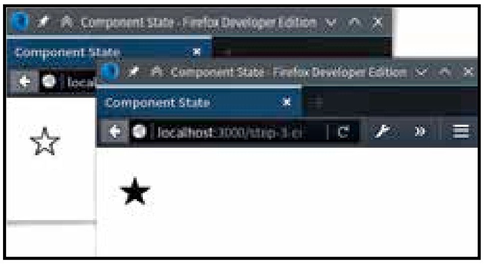
MovieList Component

While our Movie component takes a single movie and renders it, the MovieList component will take an array with data about many movies and render each using the Movie component:

```
var MovieList = React.createClass({
 render: function() {
   var movielist = this.props.movies.
map(function(item) {
     return (<li>
             <Movie
               title={item['Title']}
               year={item['Year']} />
           </li>);
   });
   return (<ul>{movielist}</ul>);
 }
});
```

In the render function this component uses the Array map method to create a Movie component for each movie data item. The reason we have the movie title set to {item['Title']} is that, that is the format in which the API we are using returns data.

The final two piece in our puzzle is a search box that we can use to perform queries. However we well also need some way to connect everything together. For that we will create yet another component, for our main app.



Once you know enough react, you could link together five star components to create a ratings widget.

The SearchBox Component

This is another simple component, but it introduces some new concepts that you will find useful. Let's have a look:

```
var SearchBox = React.createClass({
 handleSubmit: function(e) {
   e.preventDefault();
   this.props.queryChange(this.refs.query.value);
 },
 render: function() {
   return (
     <form onSubmit={this.handleSubmit}>
       <input ref="query" />
     </form>
   );
 }
});
```

As you can see from the render function, this component returns a simple form with just a single text input field. What's going on under the hood though is a lot more interesting.

Let's start with the render method, where you'll notice that we have added a ref attribute / property to the input element. You'll also notice that we have set the handleSubmit method as the submit event handler for our search form. This method will be called when someone enters some text in the input box and presses enter.

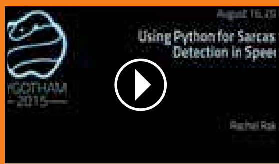
*pointers

>>Interesting developer news

*What if the user was function?

>>This great talk examines a new way of looking at user interaction by abstracting away the user itself as a function.

http://dgit.in/UserFn



*Sarcasm Detection Using Python

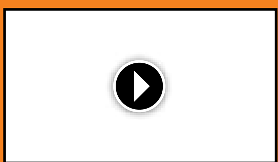
>>Are you bad at detecting sarcasm? Now let your computer help you out by building a sarcasm detector using Python.

http://dgit.in/SarcasmDtr

*Robots Running JavaScript

>>If you like Robots, and you like JavaScript, this talk is a must watch as it explores NodeBots, Robots that run on JavaScript.

http://dgit.in/NodeBots



*Rethinking Web App Development

>>This talk by a Facebook developer introduces the Flux architecture for building web applications.

http://dgit.in/FluxTalk

Interactive page